

APPENDICES

APPENDIX A — EXPERIMENTAL CONFIGURATION

A.1 Hardware Environment

- Processor: Intel Core i5-8350U @ 1.70 GHz
- RAM: 8 GB
- Operating System: Windows 11 Pro
- System architecture: 64-bit, x64-based processor

A.2 Software Environment

- Node.js: v24.x
- Express.js
- MongoDB / SQLite
- npm
- Windows/Linux environment used for testing

A.3 Iri-Shield Configuration

Mention the tested security modes:

- Low
- Medium
- High

Also mention storage configuration and relevant benchmark settings.

APPENDIX B — SECURITY TEST DATASET

Attack Category	Test Cases
SQL Injection	45
XSS	39
Path Traversal	30
Command Injection	30
SSTI	16
NoSQL Injection	8
Secret Probe	12

Scanner/Bot	15
Brute Force	8
XXE	4
LDAP Injection	4
Open Redirect	4
Header Injection	3
Base64	2
Total	220

The security dataset consisted of controlled attack scenarios representing multiple classes of API and web application attacks. The dataset was used to evaluate the detection and mitigation capabilities of Iri-Shield under controlled experimental conditions.

APPENDIX C — SECURITY EVALUATION RESULTS

Attack Category	Total	Detected	Detection Rate
SQL Injection	45	45	100%
XSS	39	39	100%
Path Traversal	30	30	100%
Command Injection	30	30	100%
SSTI	16	16	100%
NoSQL Injection	8	8	100%
Secret Probe	12	12	100%
XXE	4	4	100%
LDAP Injection	4	4	100%
Open Redirect	4	4	100%
Header Injection	3	3	100%
Base64	2	2	100%
Scanner/Bot	15	13	86.7%
Brute Force	8	4	50%
Total	220	214	97.3%

Overall, 214 of 220 controlled attack requests were detected and mitigated, resulting in an overall detection rate of 97.3%.

APPENDIX D — PERFORMANCE BENCHMARK RESULTS

D.1 Standard Benchmark

Requests	Concurrency	Baseline Req/s	Shield Req/s	Shield Avg. Latency	Shield p95
----------	-------------	----------------	--------------	---------------------	------------

300	5	1904	997	4.96 ms	7.80 ms
300	15	2404	706	21.49 ms	34.24 ms
500	5	2669	630	7.92 ms	12.16 ms
500	15	3097	730	20.44 ms	29.80 ms

D.2 High-Concurrency Benchmark

Requests	Concurrency	Baseline Req/s	Shield Req/s	Shield Avg. Latency	Shield p95
500	30	3352	760	39.25 ms	64.62 ms
1000	30	3585	814	36.67 ms	51.83 ms
2000	30	3975	786	41.91 ms	77.87 ms

APPENDIX E — FALSE POSITIVE, IDENTITY AND REDACTION RESULTS

E.1 False Positive Evaluation

Metric	Result
Legitimate Requests	2,000
False Positives	0
False Positive Rate	0.00%

You can additionally mention the separate 10,000-request terminal run if it is already documented in Chapter 5.

E.2 Identity Continuity

Scenario	Test Cases	Correct
Normal	200	200
IP Drift	100	100
Device Drift	100	100
Multi-vector	100	100
Total	500	500

Accuracy: 100%

E.3 PII Redaction

Metric	Result
Payloads Tested	500
Successful Redactions	500
Missed Sensitive Fields	0

Overmasking	0
Redaction Success	100%

APPENDIX F — SAMPLE API REQUESTS AND RESPONSES

F.1 Legitimate Request

```
GET /api/users/101
Accept: application/json
User-Agent: Mozilla/5.0
```

Expected result:

```
HTTP 200 OK
```

F.2 SQL Injection Request

```
GET /api/users?id=1' OR '1'='1
```

Expected security response:

```
HTTP 403 Forbidden
```

F.3 XSS Request

```
GET /api/search?q=<script>alert(1)</script>
```

Expected:

```
HTTP 403 Forbidden
```

Use **representative examples only**. Don't put all 220 payloads here.

APPENDIX G — IRI-SHIELD CONFIGURATION AND USAGE

Example:

```
const express = require("express");
const iriShield = require("iri-shield");

const app = express();

app.use(
```

```

    iriShield({
      mode: "high"
    })
  );

```

Then add screenshots or configuration examples that you **actually** used.

APPENDIX H — DASHBOARD AND SYSTEM SCREENSHOTS

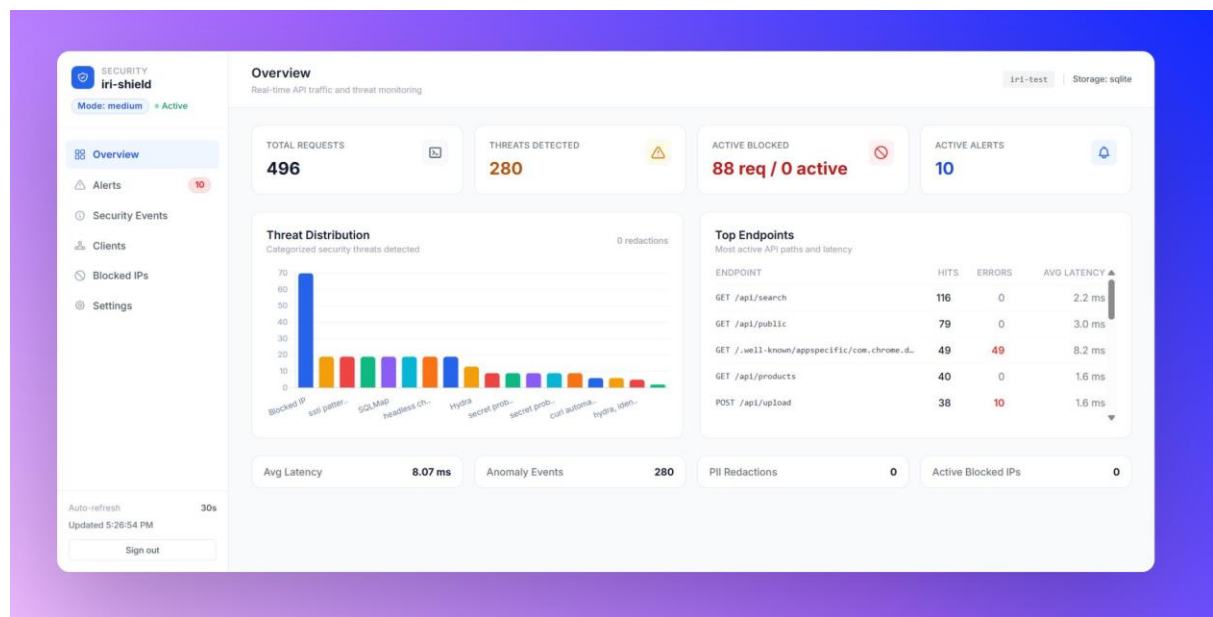


Figure H.1. Iri-Shield dashboard overview

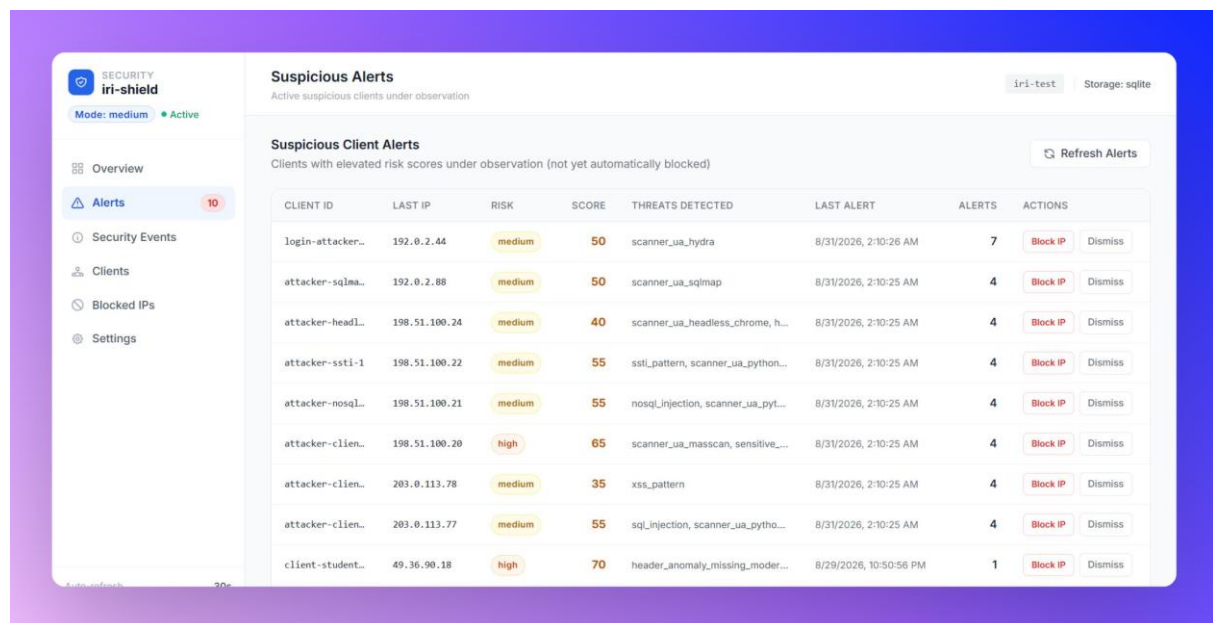


Figure H.2. Iri-Shield Security Alerts

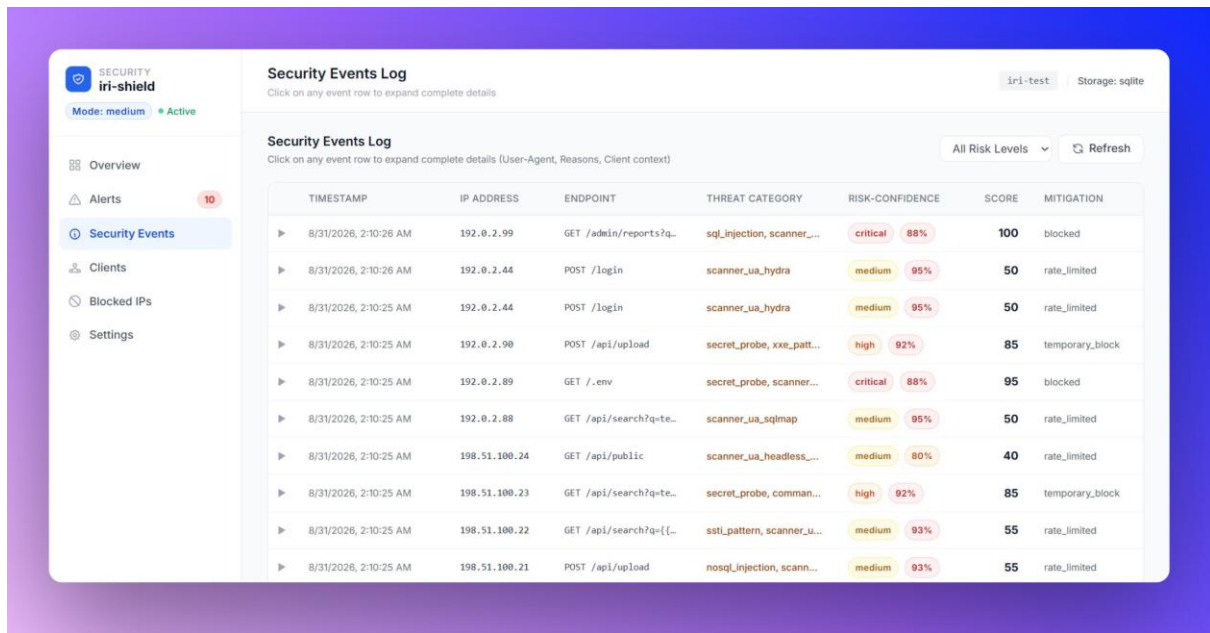


Figure H.3. Iri-Shield Security Events

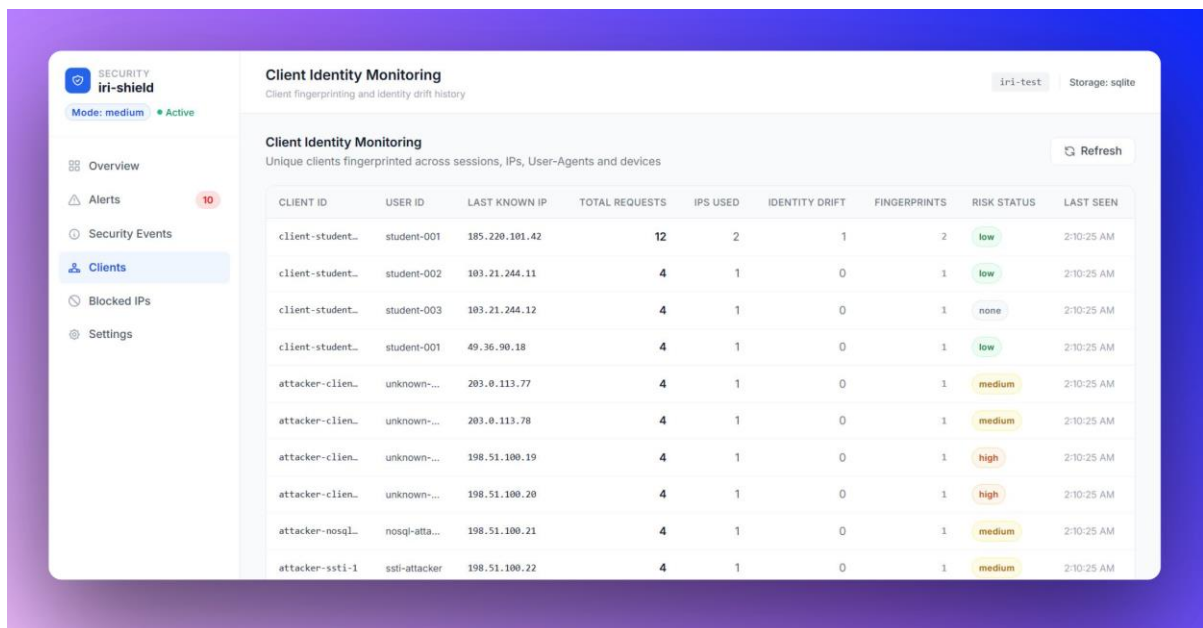


Figure H.4. Iri-Shield Client Profiling

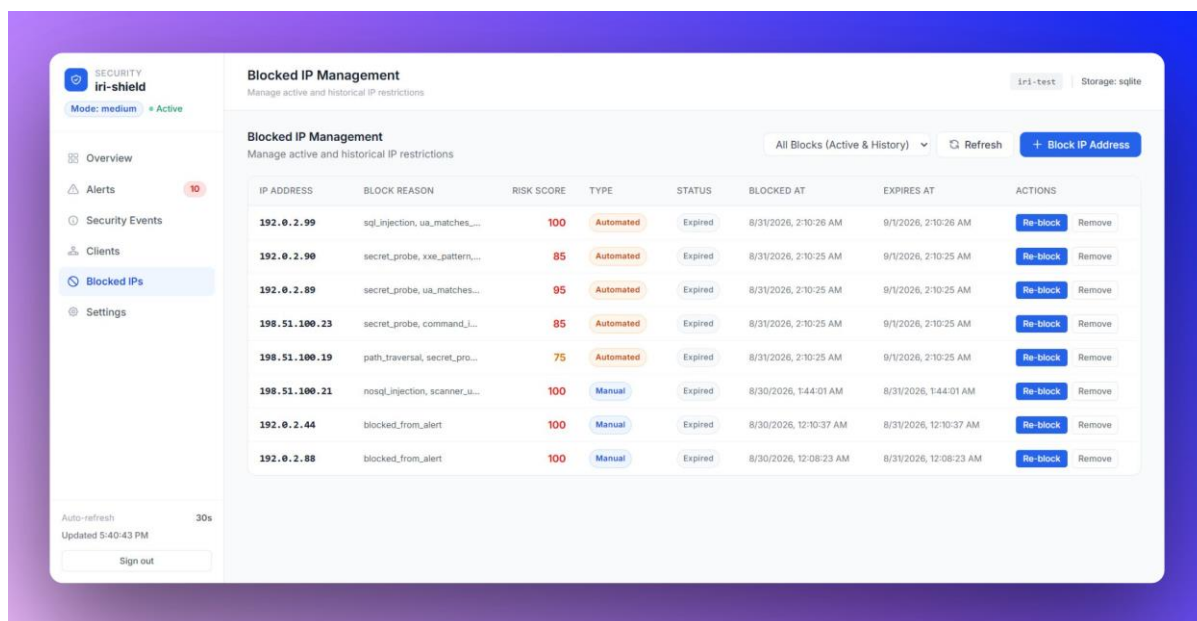


Figure H.5. Iri-Shield Blocked IPs

```
PS C:\iri-shield> npm run benchmark

> iri-shield@1.2.0 benchmark
> node benchmark/run.js

=====

iri-shield Multi-Workload Scientific Performance Benchmark Matrix
=====

Workloads Matrix : Requests [500,1000,2000] × Concurrency [5,15,30]
Repeated Trials : 3 runs per configuration (computing Mean, Median, StdDev, p95, p99)
Warm-up Phase : 150 initial requests per server to stabilize V8 JIT

[Warm-up] Warming up V8 JIT compiler on baseline (port 57302) and shield (port 57303)...
[Warm-up] Warmup completed. Beginning measured matrix execution.

Benchmarking Workload: 500 reqs @ 5 workers (3 trials)... Done! [Overhead: +3.38ms, Throughput: 2195 vs 933 req/s]
Benchmarking Workload: 500 reqs @ 15 workers (3 trials)... Done! [Overhead: +14.47ms, Throughput: 3271 vs 785 req/s]
Benchmarking Workload: 500 reqs @ 30 workers (3 trials)... Done! [Overhead: +30.12ms, Throughput: 3352 vs 760 req/s]
Benchmarking Workload: 1000 reqs @ 5 workers (3 trials)... Done! [Overhead: +6.02ms, Throughput: 3422 vs 683 req/s]
Benchmarking Workload: 1000 reqs @ 15 workers (3 trials)... Done! [Overhead: +19.87ms, Throughput: 3074 vs 619 req/s]
Benchmarking Workload: 1000 reqs @ 30 workers (3 trials)... Done! [Overhead: +28.22ms, Throughput: 3585 vs 814 req/s]
Benchmarking Workload: 2000 reqs @ 5 workers (3 trials)... Done! [Overhead: +5.18ms, Throughput: 3673 vs 775 req/s]
Benchmarking Workload: 2000 reqs @ 15 workers (3 trials)... Done! [Overhead: +12.11ms, Throughput: 4540 vs 972 req/s]
Benchmarking Workload: 2000 reqs @ 30 workers (3 trials)... Done! [Overhead: +34.05ms, Throughput: 3975 vs 786 req/s]

=====

SCIENTIFIC PERFORMANCE EVALUATION SUMMARY MATRIX
=====

Workload | Throughput (Req/s) | Avg Latency (ms) | p95 Latency (ms) | CPU Usage (%)
Reqs @ Concurr | Baseline | Shield | Baseline | Shield | Baseline | Shield | Base | Shield
-----|-----|-----|-----|-----|-----|-----|-----|-----
500 @ c=5 | 2195 | 933 | 2.34 ms | 5.72 ms | 3.7 ms | 9.29 ms | 116.8% | 108.9%
500 @ c=15 | 3271 | 785 | 4.57 ms | 19.04 ms | 6.82 ms | 29.46 ms | 133% | 104.2%
500 @ c=30 | 3352 | 760 | 9.13 ms | 39.25 ms | 13.47 ms | 64.62 ms | 104.1% | 103%
1000 @ c=5 | 3422 | 683 | 1.47 ms | 7.49 ms | 2.32 ms | 13.06 ms | 103.1% | 109.9%
1000 @ c=15 | 3074 | 619 | 4.88 ms | 24.75 ms | 7.27 ms | 41.56 ms | 99.7% | 100.7%
1000 @ c=30 | 3585 | 814 | 8.45 ms | 36.67 ms | 11.8 ms | 51.83 ms | 104.3% | 104.1%
```

```

2000 @ c=5 | 3673 | 775 | 1.38 ms | 6.56 ms | 2.13 ms | 10.59 ms | 106.3% | 101.2%
2000 @ c=15 | 4540 | 972 | 3.29 ms | 15.4 ms | 5.02 ms | 24.49 ms | 99% | 98.4%
2000 @ c=30 | 3975 | 786 | 7.86 ms | 41.91 ms | 12.79 ms | 77.87 ms | 94.4% | 98.6%
=====
Reports saved to:
- results/performance.csv
- results/comparison.json

```

Figure H.6. Iri-Shield Benchmark Output

APPENDIX I — PROJECT STRUCTURE AND ADDITIONAL ARTIFACTS

I.1 Project Directory Structure

The iri-shield project is organized into core implementation modules, research and benchmarking suites, testing resources, and generated evaluation results.

```

iri-shield/
├─ package.json
├─ index.js
├─ README.md
├─ LICENSE
├─ src/
├─ benchmark/
├─ test/
├─ results/
└─ final-results/

```

The src/ directory contains the core security middleware and supporting modules, while the benchmark/ directory contains the research evaluation and benchmarking scripts. The results/ and research-results/ directories contain generated experimental data and consolidated research outputs.

I.2 Research and Benchmark Commands

The following npm scripts were used for testing and research evaluation.

Master Research Evaluation

```
npm run research:evaluate
```

Executes the comprehensive research evaluation suite and generates consolidated research outputs.

Performance Benchmark

```
npm run benchmark
```

Runs the multi-workload performance benchmark measuring throughput, latency percentiles, and resource usage.

Security Evaluation

```
npm run security:evaluate
```

Evaluates the controlled attack dataset for category-wise threat detection and mitigation.

False-Positive Evaluation

```
npm run fp:evaluate
```

Evaluates legitimate traffic to measure the observed false-positive rate.

Identity Continuity Evaluation

```
npm run identity:evaluate
```

Evaluates client identity continuity and drift detection.

PII and Secret Redaction Evaluation

```
npm run redaction:evaluate
```

Evaluates recursive PII and secret redaction accuracy.

Additional Research Commands

```
npm run security:compare  
npm run research:charts
```

The first command performs baseline security comparison, while the second generates research visualization charts.

I.3 Test Command

The project's automated test suite can be executed using:

```
npm test
```

The command uses Node.js's built-in test runner:

```
node --test
```

I.4 Research Result Files

research-results/

This directory contains consolidated outputs generated by the research evaluation suite.

File	Description
research_summary.md	Consolidated research summary and Chapter 5-oriented analysis
experiments.csv	Master experimental metrics
summary_report.json	Machine-readable consolidated research results

results/

This directory contains detailed evaluation outputs.

File	Description
performance.csv	Performance and latency measurements
comparison.json	Performance comparison data
security.csv	Security detection results
security.json	Detailed security evaluation data
false_positives.csv	False-positive evaluation
false_positives.json	Detailed false-positive data
identity.csv	Identity continuity results
identity.json	Detailed identity evaluation data
redaction.csv	PII redaction results
redaction.json	Detailed redaction evaluation data

I.5 Selected Configuration Parameters

The following parameters represent selected implementation values used by the Iri-Shield security engine.

Composite Threat Scoring

Parameter	Default Value
SQL Injection	65
Command Injection	75
SSTI	65
Method Anomaly	20
Endpoint Flood	25
Sensitive Route Access	20
IP Change	15

User-Agent Change	20
Fingerprint Change	25
Platform Change	10
Account Takeover Chain	25
Secret Sweep	20

Security Mode Multipliers

Security Mode	Multiplier
Low	0.7
Medium	1.0
High	1.3

Identity Drift Penalties

Identity State	Points
BASELINE_NORMAL	0
IP_DRIFT	40
DEVICE_DRIFT	45
MULTI_VECTOR_ANOMALY	60

I.6 Selected Detection and Redaction Rules

Base64 Detection

The threat inspection layer identifies candidate Base64-encoded strings using:

```
[A-Za-z0-9+/]{20,}={0,2}
```

Exact-Key PII/Secret Masking

The redaction layer performs exact-key masking for sensitive fields including:

```
password
token
apiKey
```

It also performs heuristic scanning for sensitive patterns such as:

```
Email
Credit Card
Indian Aadhaar
SSN
```

I.7 Storage Configuration

SQLite storage uses WAL mode:

```
PRAGMA journal_mode = WAL;
```

The configured log retention period is:

```
retentionDays: 30
```

The middleware supports pluggable storage mechanisms including in-memory, SQLite, and MongoDB-based storage.

I.8 Package Configuration

The primary package dependencies include:

```
bcryptjs  
cors  
express  
helmet  
jsonwebtoken  
pino
```

The package also defines research-oriented npm scripts for automated benchmarking, security evaluation, false-positive testing, identity evaluation, redaction testing, and chart generation.

I.9 Reproducibility Note

The commands, project structure, configuration parameters, and result-file organization presented in this appendix are included to improve the reproducibility and transparency of the research implementation. The experimental results presented in the main chapters were generated using the documented evaluation procedures and configurations.